K-Medoids

Section 5 Spring 2026

K-Medoids is a clustering algorithm that groups similar data points together.

In K-Medoids the cluster centers (medoids) are the actual points from the data set.

How the Algorithm Works

1- Initialize: Randomly pick k actual data points to be the "starting" centers (medoids)

2- Assign points: calculate the distance and assign each point to the nearest medoid.

3- Swap (The Iteration): The algorithm tries swapping a medoid with a different point to see if it reduces the "total cost" (the sum of distances).

4- Repeat: It keeps swapping until the best representatives are found.

consider the following set of data. We will take k=2 and use the Manhattan distance formula

d = |x1 - x2| + |y1 - y2|

Point ID	X (Feature 1)	Y (Feature 2)
P_1	9	6
P_2	10	4
P_3	4	4
P_4	5	8
P_5	3	8
P_6	2	5
P_7	8	5
P_8	4	6
P_9	8	4
P_{10}	9	3

```
#install scikit-learn-extra to import KMedoids class
!pip install scikit-learn-extra
```

```
Requirement already satisfied: scikit-learn-extra in /usr/local/lib/python3.12/dist-packages (0.3.0)
Requirement already satisfied: numpy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from scikit-learn-extra) (1.26.4)
Requirement already satisfied: scipy>=0.19.1 in /usr/local/lib/python3.12/dist-packages (from scikit-learn-extra) (1.16.3)
Requirement already satisfied: scikit-learn>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn-extra) (1.
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.23.0->scikit-l
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.23.0->s
```

```
!pip install numpy==1.26.4
```

```
Requirement already satisfied: numpy==1.26.4 in /usr/local/lib/python3.12/dist-packages (1.26.4)
```

```
import numpy as np
import pandas as pd
```

```
import matplotlib.pyplot as plt
from sklearn_extra.cluster import KMedoids
```

```
#read data in 2d array
data = np.array([[9,6],[10,4],[4,4],[5,8],[3,8],[2,5],[8,5],[4,6],[8,4],[9,3]])
```

```
#create instance from KMedoids
# we can control the maximum number of iterations using the max_iter parameter
k=2
Kmedoids_model = KMedoids(n_clusters=k, metric="manhattan",init="random")
```

```
#fitting model
Kmedoids_model.fit(data)
```

```
KMedoids
KMedoids(init='random', metric='manhattan', n_clusters=2)
```

```
labels= Kmedoids_model.labels_
labels
```

```
array([1, 1, 0, 0, 0, 0, 1, 0, 1, 1])
```

```
centers=Kmedoids_model.cluster_centers_
print("centers", centers)
```

```
centers [[4. 6.]
 [8. 4.]]
```

```
#medoids indices
Kmedoids_model.medoid_indices_
```

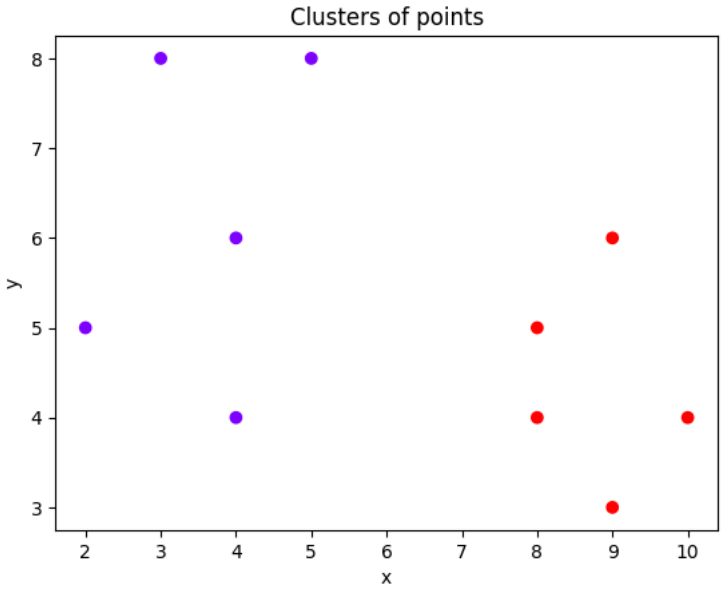
```
array([7, 8])
```

```
# print the points for each center

# Loop over each cluster
for j in range(k):
# Print which cluster is displaying
    print("\nCluster", j)
# Boolean masking: select all points in 'data' where the label equals current cluster 'j'
    print(data[labels == j])
```

```
Cluster 0
[[4 4]
 [5 8]
 [3 8]
 [2 5]
 [4 6]]

Cluster 1
[[ 9  6]
 [10  4]
 [ 8  5]
 [ 8  4]
 [ 9  3]]
```

```
#scattering
plt.scatter(data[:,0], data[:,1], c=Kmedoids_model.labels_, cmap='rainbow')
plt.title('Clusters of points')
plt.xlabel('x')
plt.ylabel('y')
```

```
Text(0, 0.5, 'y')
```




```
# Example of a new data point
new_point = np.array([[3, 6]])

predicted_label = Kmedoids_model.predict(new_point)
print("Predicted cluster for the new point:", predicted_label)
```

```
Predicted cluster for the new point: [0]
```

```
#read data file
shopping_data= pd.read_csv('Mall_Customers_dataset.csv')
shopping_data
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
...	...	...	...	...	...
95	96	Male	24	60	52
96	97	Female	47	60	47
97	98	Female	27	60	50
98	99	Male	48	61	42
99	100	Male	20	61	49

100 rows × 5 columns

Next steps: Generate code with shopping_data New interactive sheet

```
# needed features
data = shopping_data.iloc[:, 3:5].values
```

```
#create object from KMedoids class
cluster = KMedoids(n_clusters=3, metric="manhattan",init="random") #random_state=33

cluster.fit_predict(data)
```

```
array([0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1,
       0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 2, 1, 2, 2,
       0, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

```
#scattering data
plt.scatter(data[:,0], data[:,1], c=cluster.labels_, cmap='rainbow')
plt.title('Clusters of customers')
plt.xlabel('Annual Income ($)')
plt.ylabel('Spending Score (1-100)')
```

```
Text(0, 0.5, 'Spending Score (1-100)')
```